

Predictive Analysis: Problem Sets 6 & 7

Dipankar Sahu (727)

Problem Set 6: Model Selection (Subset Selection Methods)

```
# 1. Clean data - remove rows with missing values
data(Hitters)
Hitters <- na.omit(Hitters)
cat("Dimensions after cleaning:", dim(Hitters), "\n")

## Dimensions after cleaning: 263 20

# 2. Train-test split (80-20)
set.seed(42)
n <- nrow(Hitters)
train_idx <- sample(1:n, size = floor(0.8 * n))
train <- Hitters[train_idx, ]
test <- Hitters[-train_idx, ]
cat("Training size:", nrow(train), "| Test size:", nrow(test), "\n")

## Training size: 210 | Test size: 53

# Helper function: compute MSE from regsubsets model
predict_regsubsets <- function(object, newdata, id) {
  form <- as.formula(object$call[[2]])
  mat <- model.matrix(form, newdata)
  coefi <- coef(object, id = id)
  mat[, names(coefi)] %*% coefi
}

get_mse <- function(object, traindata, testdata, nvars) {
  train_mat <- model.matrix(Salary ~ ., traindata)
  test_mat <- model.matrix(Salary ~ ., testdata)

  sapply(nvars, function(i) {
    coefi <- coef(object, id = i)
    train_pred <- train_mat[, names(coefi)] %*% coefi
    test_pred <- test_mat[, names(coefi)] %*% coefi
    c(train_mse = mean((traindata$Salary - train_pred)^2),
      test_mse = mean((testdata$Salary - test_pred)^2))
  })
}
```

```

# Function to select best model per criterion and report MSEs
report_best <- function(reg_obj, train_df, test_df, method_name) {
  s <- summary(reg_obj)

  best_adjr2 <- which.max(s$adjr2)
  best_bic <- which.min(s$bic)
  best_cp <- which.min(s$cp)
  # AIC not directly from leaps; approximate via BIC-like formula or use Cp (
  proportional)
  #  $AIC \sim n \cdot \log(RSS/n) + 2p$ ; we compute manually
  n_tr <- nrow(train_df)
  aic_vals <- n_tr * log(s$rss / n_tr) + 2 * (1:19 + 1)
  best_aic <- which.min(aic_vals)

  criteria <- c(AdjR2 = best_adjr2, BIC = best_bic, AIC = best_aic, Cp = best
_cp)
  cat("\n===", method_name, "===\n")
  cat("Best model size by criterion:\n"); print(criteria)

  results <- data.frame(Criterion = names(criteria), Best_Size = as.integer(c
riteria),
                        Train_MSE = NA, Test_MSE = NA)

  train_mat <- model.matrix(Salary ~ ., train_df)
  test_mat <- model.matrix(Salary ~ ., test_df)

  for (i in seq_len(nrow(results))) {
    k <- results$Best_Size[i]
    coefs <- coef(reg_obj, id = k)
    tr_p <- train_mat[, names(coefs)] %*% coefs
    te_p <- test_mat[, names(coefs)] %*% coefs
    results$Train_MSE[i] <- mean((train_df$Salary - tr_p)^2)
    results$Test_MSE[i] <- mean((test_df$Salary - te_p)^2)
  }

  print(results)
  invisible(results)
}

```

3-4. Best Subset Selection

```

reg_best <- regsubsets(Salary ~ ., data = train, nvmax = 19, method = "exhaus
tive")
res_best <- report_best(reg_best, train, test, "Best Subset Selection")

##
## === Best Subset Selection ===
## Best model size by criterion:
## AdjR2   BIC   AIC   Cp
##      8     5     6     6

```

```
## Criterion Best_Size Train_MSE Test_MSE
## 1 AdjR2 8 73661.15 208369.1
## 2 BIC 5 76053.42 235199.6
## 3 AIC 6 74817.19 229808.5
## 4 Cp 6 74817.19 229808.5
```

5. Forward Stepwise Selection

```
reg_fwd <- regsubsets(Salary ~ ., data = train, nvmax = 19, method = "forward")
res_fwd <- report_best(reg_fwd, train, test, "Forward Stepwise Selection")

##
## === Forward Stepwise Selection ===
## Best model size by criterion:
## AdjR2 BIC AIC Cp
## 10 5 10 10
## Criterion Best_Size Train_MSE Test_MSE
## 1 AdjR2 10 72930.58 199968.8
## 2 BIC 5 79565.26 215218.6
## 3 AIC 10 72930.58 199968.8
## 4 Cp 10 72930.58 199968.8
```

6. Backward Stepwise Selection

```
reg_bwd <- regsubsets(Salary ~ ., data = train, nvmax = 19, method = "backward")
res_bwd <- report_best(reg_bwd, train, test, "Backward Stepwise Selection")

##
## === Backward Stepwise Selection ===
## Best model size by criterion:
## AdjR2 BIC AIC Cp
## 10 5 6 6
## Criterion Best_Size Train_MSE Test_MSE
## 1 AdjR2 10 72930.58 199968.8
## 2 BIC 5 76053.42 235199.6
## 3 AIC 6 75054.99 230585.3
## 4 Cp 6 75054.99 230585.3
```

7. Comparison of Three Methods

```
cat("\n=== Summary Comparison ===\n")

##
## === Summary Comparison ===

cat("\nBest Subset:\n"); print(res_best)

##
## Best Subset:
```

```
## Criterion Best_Size Train_MSE Test_MSE
## 1 AdjR2 8 73661.15 208369.1
## 2 BIC 5 76053.42 235199.6
## 3 AIC 6 74817.19 229808.5
## 4 Cp 6 74817.19 229808.5

cat("\nForward Stepwise:\n"); print(res_fwd)

##
## Forward Stepwise:

## Criterion Best_Size Train_MSE Test_MSE
## 1 AdjR2 10 72930.58 199968.8
## 2 BIC 5 79565.26 215218.6
## 3 AIC 10 72930.58 199968.8
## 4 Cp 10 72930.58 199968.8

cat("\nBackward Stepwise:\n"); print(res_bwd)

##
## Backward Stepwise:

## Criterion Best_Size Train_MSE Test_MSE
## 1 AdjR2 10 72930.58 199968.8
## 2 BIC 5 76053.42 235199.6
## 3 AIC 6 75054.99 230585.3
## 4 Cp 6 75054.99 230585.3
```

Interpretation: All three methods tend to select similar models for most criteria (especially BIC and Cp which favor parsimony). Forward and Backward stepwise are computationally cheaper than Best Subset but may differ slightly in selected variables. Test MSE is the most reliable guide to generalization performance.

Problem Set 7: Shrinkage Methods

```
if (!require(glmnet)) install.packages("glmnet")
library(glmnet)

# Fresh clean data
data(Hitters)
Hitters <- na.omit(Hitters)

# 1. Split: 100 test, rest training
set.seed(42)
test_idx <- sample(1:nrow(Hitters), 100)
train7 <- Hitters[-test_idx, ]
test7 <- Hitters[test_idx, ]
```

```

X_train <- model.matrix(Salary ~ ., train7)[, -1]
y_train <- train7$Salary
X_test  <- model.matrix(Salary ~ ., test7)[, -1]
y_test  <- test7$Salary

cat("Train size:", nrow(train7), "| Test size:", nrow(test7), "\n")

## Train size: 163 | Test size: 100

```

2. Least Squares

```

ols_fit  <- lm(Salary ~ ., data = train7)
ols_train <- predict(ols_fit, train7)
ols_test  <- predict(ols_fit, test7)

ols_train_mse <- mean((y_train - ols_train)^2)
ols_test_mse  <- mean((y_test  - ols_test)^2)

cat("OLS Train MSE:", round(ols_train_mse, 2), "\n")

## OLS Train MSE: 91923.29

cat("OLS Test MSE: ", round(ols_test_mse, 2), "\n")

## OLS Test MSE: 124714

```

3. Ridge Regression

```

# Grid of Lambda values
lambda_grid <- 10^seq(10, -2, length = 100)

ridge_fit <- glmnet(X_train, y_train, alpha = 0, lambda = lambda_grid)

# (i) Dimension of coefficient matrix
cat("Coefficient matrix dimensions:", dim(coef(ridge_fit)), "\n")

## Coefficient matrix dimensions: 20 100

# (ii) L2 norm at 50th and 60th Lambda
l2_50 <- sqrt(sum(coef(ridge_fit)[-1, 50]^2))
l2_60 <- sqrt(sum(coef(ridge_fit)[-1, 60]^2))
cat("L2 norm at lambda[50] =", round(lambda_grid[50], 4), ":", round(l2_50, 4), "\n")

## L2 norm at lambda[50] = 11497.57 : 7.6918

cat("L2 norm at lambda[60] =", round(lambda_grid[60], 4), ":", round(l2_60, 4), "\n")

## L2 norm at lambda[60] = 705.4802 : 65.8542

cat("Larger lambda => smaller L2 norm (more shrinkage)\n")

```

```
## Larger lambda => smaller L2 norm (more shrinkage)

# (iv) Test MSE for 50th and 60th Lambda
pred_50 <- predict(ridge_fit, s = lambda_grid[50], newx = X_test)
pred_60 <- predict(ridge_fit, s = lambda_grid[60], newx = X_test)
cat("Test MSE at lambda[50]:", round(mean((y_test - pred_50)^2), 2), "\n")

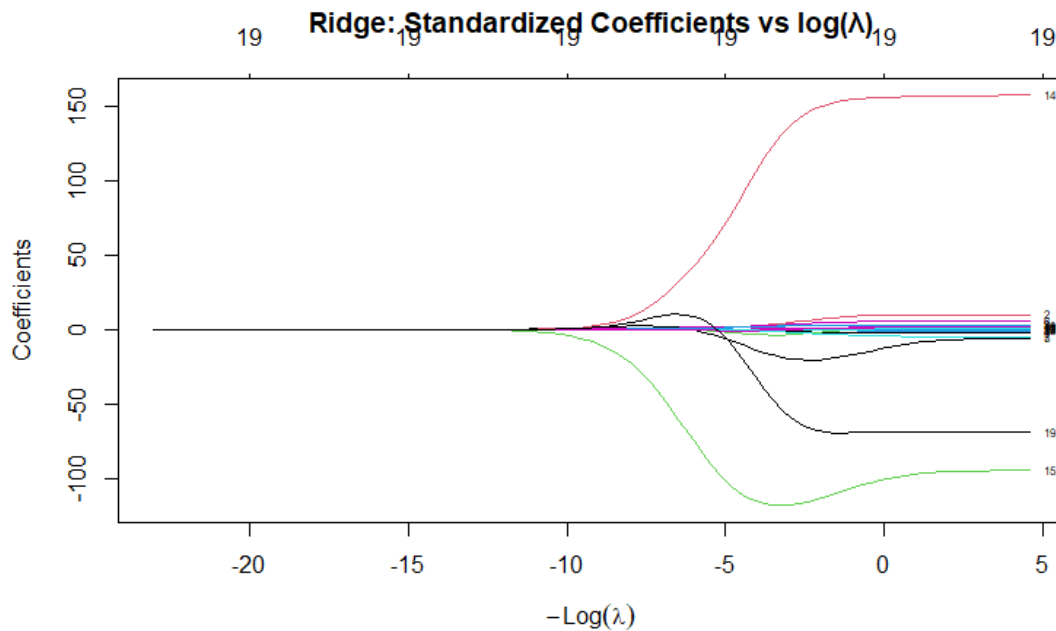
## Test MSE at lambda[50]: 117860.6

cat("Test MSE at lambda[60]:", round(mean((y_test - pred_60)^2), 2), "\n")

## Test MSE at lambda[60]: 85167.17
```

4. Ridge Coefficient Path Plot (vs λ)

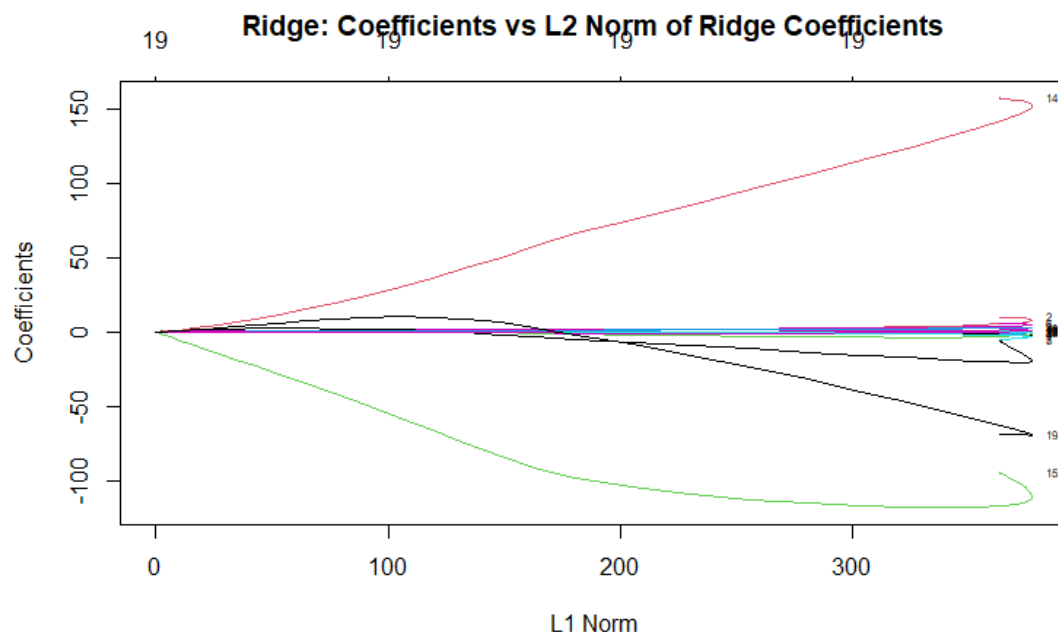
```
plot(ridge_fit, xvar = "lambda", label = TRUE,
     main = "Ridge: Standardized Coefficients vs log( $\lambda$ )")
```



Interpretation: As λ increases, all coefficients shrink toward zero. No coefficient reaches exactly zero — Ridge retains all predictors but with reduced magnitudes.

5. Coefficient Path vs L2 Norm Ratio

```
# Plot using xvar = "norm" which shows L2 norm of ridge coef
plot(ridge_fit, xvar = "norm", label = TRUE,
     main = "Ridge: Coefficients vs L2 Norm of Ridge Coefficients")
```



Interpretation: As we move left (smaller L2 norm = higher λ), all coefficients shrink toward zero. This is equivalent to the λ plot but reversed on the x-axis.

6. Ridge with Cross-Validation

```
set.seed(42)
cv_ride <- cv.glmnet(X_train, y_train, alpha = 0)
best_lambda_ride <- cv_ride$lambda.min
cat("Best lambda (CV):", round(best_lambda_ride, 4), "\n")

## Best lambda (CV): 29.8318

ridge_pred_cv <- predict(cv_ride, s = best_lambda_ride, newx = X_test)
ridge_test_mse <- mean((y_test - ridge_pred_cv)^2)
cat("Ridge Test MSE (CV lambda):", round(ridge_test_mse, 2), "\n")

## Ridge Test MSE (CV lambda): 104451.5
```

7. LASSO with Cross-Validation

```
set.seed(42)
cv_lasso <- cv.glmnet(X_train, y_train, alpha = 1)
best_lambda_lasso <- cv_lasso$lambda.min
cat("Best lambda (CV):", round(best_lambda_lasso, 4), "\n")

## Best lambda (CV): 2.8476

lasso_pred <- predict(cv_lasso, s = best_lambda_lasso, newx = X_test)
lasso_test_mse <- mean((y_test - lasso_pred)^2)

lasso_coefs <- coef(cv_lasso, s = best_lambda_lasso)
```

```

nonzero      <- sum(lasso_coefs != 0) - 1 # exclude intercept
cat("LASSO Test MSE:", round(lasso_test_mse, 2), "\n")

## LASSO Test MSE: 116353.6

cat("Number of non-zero coefficients:", nonzero, "\n")

## Number of non-zero coefficients: 17

```

8. Elastic Net with Cross-Validation

```

# Try several alpha values and pick the best via CV
alphas      <- seq(0.1, 0.9, by = 0.1)
cv_errors   <- sapply(alphas, function(a) {
  cv_fit <- cv.glmnet(X_train, y_train, alpha = a)
  min(cv_fit$cvm)
})

best_alpha  <- alphas[which.min(cv_errors)]
cat("Best alpha:", best_alpha, "\n")

## Best alpha: 0.7

set.seed(42)
cv_enet     <- cv.glmnet(X_train, y_train, alpha = best_alpha)
best_lam_en <- cv_enet$lambda.min
cat("Best lambda (CV):", round(best_lam_en, 4), "\n")

## Best lambda (CV): 3.3773

enet_pred    <- predict(cv_enet, s = best_lam_en, newx = X_test)
enet_test_mse <- mean((y_test - enet_pred)^2)

enet_coefs   <- coef(cv_enet, s = best_lam_en)
nonzero_en   <- sum(enet_coefs != 0) - 1
cat("Elastic Net Test MSE:", round(enet_test_mse, 2), "\n")

## Elastic Net Test MSE: 116299

cat("Non-zero predictors:", nonzero_en, "\n")

## Non-zero predictors: 17

cat("Non-zero predictor names:\n")

## Non-zero predictor names:

print(rownames(enet_coefs)[enet_coefs[,1] != 0 & rownames(enet_coefs) != "(Intercept)"])

## [1] "AtBat"      "Hits"      "HmRun"     "RBI"      "Walks"
## [6] "Years"     "CHits"     "CHmRun"    "CRuns"    "CRBI"

```

```
## [11] "CWalks"      "LeagueN"      "DivisionW"    "PutOuts"      "Assists"
## [16] "Errors"      "NewLeagueN"
```

9. Final Comparison

```
comparison <- data.frame(
  Method      = c("OLS", "Ridge (CV)", "LASSO (CV)", "Elastic Net (CV)"),
  Test_MSE    = round(c(ols_test_mse, ridge_test_mse, lasso_test_mse, enet_test_mse), 2)
)
print(comparison)

##           Method Test_MSE
## 1           OLS 124714.0
## 2      Ridge (CV) 104451.5
## 3      LASSO (CV) 116353.6
## 4 Elastic Net (CV) 116299.0
```

Interpretation: Shrinkage methods (Ridge, LASSO, Elastic Net) typically outperform OLS on test data because they reduce overfitting. LASSO also performs variable selection (many zero coefficients), making the model more interpretable. Elastic Net combines Ridge and LASSO penalties and can outperform either alone. The method with the lowest test MSE gives the most accurate salary predictions.